

Projetos Computacionais: da Teoria à Prática



Conteudista: Prof. Dr. Artur Marques

Revisão Textual: M.^a Bruna Giovana Bengozi

Objetivo da Unidade:

- Integrar as competências adquiridas e desenvolver soluções computacionais aplicáveis a problemas reais.



Material Teórico



Material Complementar

DESAFIO



Situação-Problema 1



Situação-Problema 2



Situação-Problema 3

ATIVIDADE

 Atividade de Entrega**REFERÊNCIAS**

 Referências

Material Teórico

Olá, estudante!

Vamos iniciar a disciplina abordando os conceitos necessários para que você possa realizar a atividade através das situações-problema mais à frente.



Atenção, estudante! Aqui, reforçamos o acesso ao conteúdo *online* para que você assista à videoaula. Será muito importante para o entendimento do conteúdo.

Aprendizagem Baseada em Projetos (ABP)

A Aprendizagem Baseada em Projetos (ABP) vai além da simples concepção de um mero método de ensino para se configurar como uma abrangente estratégia pedagógica. Seu ponto focal reside na ideia de que os(as) estudantes adquirem conhecimentos e habilidades de forma mais significativa e duradoura quando engajados ativamente na investigação e resolução de problemas, culminando na criação de um produto, apresentação ou performance tangível.

Conforme elucidam Larmer, Mergendoller e Boss (2015, p. 6), a ABP é um "método de ensino no qual os alunos aprendem através do engajamento ativo em projetos do mundo real e pessoalmente significativos". Essa abordagem pedagógica coloca o discente no centro do processo de aprendizagem, promovendo autonomia, responsabilidade e colaboração. Diferentemente de modelos tradicionais, nos quais o(a) professor(a) é o(a) detentor(a) primário(a) do conhecimento e o transmite de forma expositiva, na ABP, o(a) docente assume o papel de facilitador(a), orientador(a) e mediador(a) do processo de descoberta e de construção do saber pelo(a) aluno(a).

A eficácia da ABP sustenta-se em alguns princípios e características cruciais que a distinguem, como o foco em questões ou problemas desafiadores, ou seja, os projetos em ABP geralmente partem de uma questão norteadora ou de um problema complexo que não possui uma solução única ou simples. Essa questão deve ser instigante e relevante para os(as) estudantes, incentivando a investigação aprofundada (Thomas, 2000). Além disso, é possível citar a investigação sustentada, na qual os(as) alunos se engajam em um processo rigoroso e estendido

de investigação, buscando informações, aplicando conceitos e desenvolvendo soluções ao longo de um período significativo. Isso envolve pesquisa, análise crítica e síntese de informações de diversas fontes.

Outros pontos relevantes levam em consideração a autenticidade fortemente contextualizados no mundo real, abordando problemas relevantes para os(as) discentes, suas comunidades ou o campo profissional. Essa autenticidade aumenta o engajamento e a percepção da relevância do aprendizado (Blumenfeld *et al.*, 1991).

Além disso, não devemos ignorar o espaço para o protagonismo do(a) aluno(a), que pode fazer escolhas significativas sobre o projeto, seja no tema, nos métodos de investigação, nos produtos a serem desenvolvidos ou na forma de apresentação. É fundamental que, ao longo do projeto, os(as) estudantes sejam incentivados(a) a refletirem sobre o que e como estão aprendendo, bem como sobre o próprio processo de trabalho. A reflexão, assim, ajuda a consolidar o conhecimento e a desenvolver habilidades metacognitivas.

No contexto da Ciência da Computação, essa metodologia se revela particularmente vantajosa. A natureza intrinsecamente prática e orientada à resolução de problemas dessa área do conhecimento encontra na ABP um alinhamento natural. Algumas vantagens se destacam:

- Desenvolvimento de habilidades técnicas aplicadas;
- Fomento de *soft skills*;
- Aumento da motivação e engajamento;

- Aprendizagem profunda e significativa;
- Preparação para o mercado de trabalho.

Mas não podemos perder a perspectiva do papel do(a) aluno(a), que deixa de ser um(a) receptor(a) passivo(a) de informações para se tornar o(a) protagonista de seu aprendizado. Portanto, esperamos de sua parte uma postura:

- Ativa e investigativa;
- Colaborativa;
- Autônoma e responsável;
- Reflexiva.

Cabe um comentário sobre o termo “colaborativa”: embora o trabalho deva ser uma entrega individual, nada impede que os(as) alunos(as) conversem entre si e compartilhem experiências. Essa troca ajuda a elevar significativamente a qualidade de todos os trabalhos produzidos.

Refleta

A nossa disciplina PIT em Ciência da Computação foi concebida como um espaço para a articulação entre teoria e prática. Seu principal objetivo, como delineado no plano de ensino, é integrar as competências adquiridas e desenvolver soluções computacionais aplicáveis a problemas reais. Para alcançar tal objetivo, fundamentamo-nos integralmente na metodologia ABP.

Nossas atividades serão majoritariamente práticas, intercaladas com momentos de exposição teórica para embasar os conceitos essenciais já abordados em nosso material teórico.

O cerne desta disciplina é o desenvolvimento de um projeto computacional simples e factível, compatível com o estágio inicial da sua formação em Ciência da Computação. O tema específico do projeto foi pensado para ser ao mesmo tempo útil e interessante, com potencial de uso pessoal ou por outras pessoas. Ainda assim, o principal objetivo do projeto é promover a aplicação dos seguintes conhecimentos:

- Linguagem de programação: Java.
- Paradigma de programação: orientação a objetos.

O sistema que será desenvolvido será explicado no tópico a seguir.

Catálogo Simples de Livros ou Filmes

- **Descrição**

Uma aplicação web que permita ao(à) usuário(a) catalogar e consultar informações sobre livros, séries ou filmes. Cada item deve conter atributos mínimos, como título, autor/diretor, ano de publicação/lançamento, gênero e uma breve sinopse.

Funcionalidades mandatórias essenciais:

- **Funcionalidades mandatórias essenciais:**
 - Interface web para navegação e gerenciamento;
 - Cadastro de novos livros/filmes;
 - Listagem de todos os itens catalogados;

- Visualização dos detalhes de um item específico;
 - Opção de editar as informações de um item;
 - Opção de excluir um item do catálogo;
 - Busca simples por título ou autor/diretor;
 - Persistência dos dados em banco de dados.
- **Principais aprendizados envolvidos:**
 - Modelagem da entidade Livro ou Filme;
 - Implementação de CRUD (*Create, Read, Update, Delete*);
 - Desenvolvimento de JSPs para exibição de lista, detalhes e formulários;
 - Implementação de uma funcionalidade de busca básica usando SQL.

Sua complexidade de desenvolvimento é muito baixa e envolve uma entidade principal com atributos claros. A funcionalidade de busca simples adiciona um pequeno desafio interessante voltada para o SQL.

- **Possíveis extensões (caso haja tempo e interesse da sua parte — não é obrigatório —, essa etapa dependerá unicamente do seu desejo de se superar e ir além do esperado):**
 - Adicionar um sistema de avaliação (use estrelas ou emoji que desejar);
 - Permitir o upload de uma imagem de capa (requer cuidado com o manuseio de arquivos — se for muito complexo para você, pode ser simplificado para um simples link externo com a imagem);
 - Filtragem por gênero (romance, drama, comédia, ficção, ação, aventura, documentário etc.).

Uma dica importante: é fundamental que o escopo do projeto seja gerenciável dentro do cronograma semestral. A ideia não é construir um sistema comercial complexo, mas, sim, vivenciar o ciclo de desenvolvimento de um software em menor escala, aplicando os conceitos aprendidos.

Os componentes de projeto que você seguirá para a construção deverá contemplar e documentar as seguintes fases:

- **Fase de concepção e planejamento:** definição detalhada do escopo, levantamento de requisitos, modelagem inicial do sistema (casos de uso, diagramas de classes simplificados, modelo de dados) e planejamento das entregas;
- **Fase de desenvolvimento (iterativo):** implementação dos módulos do sistema;
- **Fase de testes e refinamento:** testes unitários e de integração, bem como correção de falhas;
- **Fase de documentação e preparação para apresentação:** elaboração do relatório técnico e dos materiais para a apresentação final documentada do projeto.

Vale reforçar que o projeto será desenvolvido individualmente. Este projeto integrador não é apenas um trabalho a ser entregue, mas uma jornada de aprendizado. Encare-o como uma oportunidade para aplicar seus conhecimentos, desenvolver novas habilidades, enfrentar desafios e, acima de tudo, aprender fazendo. Assim você se preparará para o mercado supercompetitivo que estamos vivenciando hoje.

Isso é muito importante, pois vivemos em um século que impõe uma reconfiguração nas demandas da sociedade e, conseqüentemente, nas competências requeridas dos(as) profissionais em todas as áreas, com especial ênfase nos campos tecnológicos, como a Ciência da Computação. Já não basta dominar apenas o conhecimento técnico específico; é imperativo desenvolver um conjunto de habilidades transversais, frequentemente denominadas competências essenciais, que são cruciais para a adaptação, inovação e resolução de problemas em um mundo cada vez mais complexo, interconectado e dinâmico.

Trilling e Fadel (2009, p. 47) argumentam que “para ter sucesso na vida e no trabalho do século XXI, os cidadãos e trabalhadores devem ser proficientes em habilidades do século XXI, como pensamento crítico, resolução de problemas, comunicação e colaboração”. A Partnership for 21st Century Skills, a P21, uma organização influente na disseminação dessas ideias, categoriza essas habilidades em diversos eixos, sendo os “4 Cs” — Pensamento Crítico (*Critical Thinking*), Criatividade (*Creativity*), Colaboração (*Collaboration*) e Comunicação (*Communication*), amplamente reconhecidos como pilares centrais.

Fundamentos de Programação em Java

Neste item, iremos retomar os pilares da linguagem de programação Java, essencial para o desenvolvimento do projeto integrador desta disciplina. O domínio desses fundamentos é condição mandatória para a construção de aplicações robustas e eficientes. Vamos relembrar desde a configuração do ambiente até os conceitos avançados de orientação a objetos e tratamento de exceções, fornecendo uma base sólida para as etapas subsequentes de desenvolvimento.

A linguagem Java foi concebida pela Sun Microsystems no início da década de 1990 e, atualmente, é mantida pela Oracle, consolidando-se como uma das mais proeminentes e versáteis no panorama da Ciência da Computação. Sua popularidade deriva de características como a portabilidade, viabilizada pela Máquina Virtual Java ou JVM, que permite a execução de código Java compilado conhecido como bytecode em diversas plataformas sem a necessidade de recompilação. É intrinsecamente orientada a objetos, robusta e possui um vasto ecossistema de bibliotecas e *frameworks*. Conforme Deitel e Deitel (2017, p. 2), “Java é uma linguagem de programação orientada a objetos poderosa e amplamente utilizada no desenvolvimento de aplicações corporativas e mobile”.

Para iniciar o desenvolvimento em Java, é imprescindível a instalação do Java Development Kit JDK, a JVM e outras ferramentas essenciais. O ambiente de desenvolvimento integrado IDE, como Eclipse, IntelliJ IDEA ou Apache NetBeans, facilita sobremaneira o processo de codificação, depuração e gerenciamento de projetos. Recomendamos fortemente que faça os downloads e instale em seu equipamento.

Aqui, oferecemos a você alguns recursos por meio dos links estáveis da web para que se aprofunde, aprenda ou recicle o seu conhecimento a fim de que o desenvolvimento do sistema proposto saia sem muitas intercorrências.

Leitura

Sites Importantes para Utilização da Linguagem Java

Oracle Java Tutorials – *Getting Started*

Clique no botão para conferir o conteúdo.

ACESSE

Instalação do JDK e Configuração do Ambiente (W3Schools)

Clique no botão para conferir o conteúdo.

ACESSE

A sintaxe de Java possui similaridades com C/C++. O entendimento de seus elementos básicos é o primeiro passo para a escrita de programas funcionais.

Java possui tipos de dados primitivos para representar valores numéricos inteiros como int, byte, short, long; e de ponto flutuante como float, double, caracteres char e booleanos boolean. Variáveis são declaradas com um tipo e um nome, bem como podem ser inicializadas. A linguagem oferece operadores aritméticos (+, -, *, /, %), relacionais (==, !=, >, <, >=, <=), lógicos

(&&, ||, !) e de atribuição (=, +=, etc.). Além disso, permite o controle do fluxo de execução com base em condições. As principais são if, if-else, if-else-if e switch. Possui também instruções para executar blocos de código múltiplas vezes. Incluem instruções for, que são ideais para iterações com contagem definida, while, que executa enquanto uma condição for verdadeira, e do-while, que executa ao menos uma vez e depois verifica a condição.

Leitura

Instruções Importantes

Java Basic Syntax (TutorialsPoint)

Clique no botão para conferir o conteúdo.

ACESSE

Java Control Flow Statements (Oracle Docs)

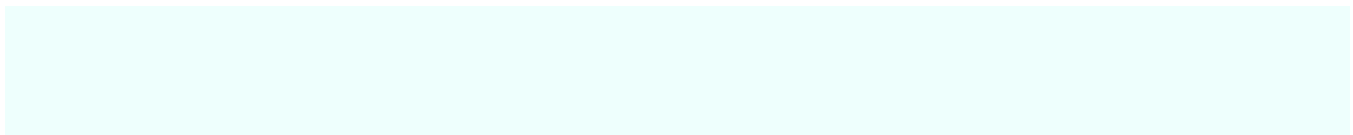
Clique no botão para conferir o conteúdo.

ACESSE

Programação Orientada a Objetos (POO) em Java

A Programação Orientada a Objetos (POO) é um paradigma que organiza o software em torno de objetos, que são instâncias de classes. Isso promove a modularidade, reutilização e manutenibilidade do código. Segundo Booch *et al.* (2007, p. 41), a orientação a objetos é um método de modelagem de sistemas complexos como um conjunto de objetos que interagem entre si.

- **Classes e objetos:** uma classe é um molde ou projeto que define os atributos que, para nós, são os dados e métodos que descrevem os comportamentos que seus objetos terão. Um objeto é uma instância concreta de uma classe;
- **Atributos e métodos:** atributos, também chamados de campos ou variáveis de instância, armazenam o estado de um objeto. Os métodos, por outro lado, definem as ações que um objeto pode realizar;
- **Encapsulamento:** é um princípio da POO que consiste em agrupar dados, especialmente os atributos, e os métodos que os manipulam dentro de uma unidade que chamamos de classe. Esse conceito ajuda a proteger os dados contra acesso externo indevido. A visibilidade de atributos e métodos é controlada por modificadores de acesso: *public*, *private*, *protected* e *default*;
- **Métodos especiais – Construtores:** são métodos com o mesmo nome da classe, responsáveis por inicializar os objetos no momento em que são criados. Dentro desses métodos (e de outros métodos da classe), pode-se usar a palavra-chave *this*, que faz referência à instância atual do objeto, permitindo acessar seus próprios métodos ou construtores.



Site

Reciclagem do Conhecimento

Object-Oriented Programming Concepts (Oracle Docs)

Clique no botão para conferir o conteúdo.

ACESSE

Leitura

Java Classes and Objects (W3Schools)

Clique no botão para conferir o conteúdo.

ACESSE

Agora que você já reciclou a parte fundamental do Java, iremos um pouco mais adiante com mecanismos que conferem maior flexibilidade e poder à modelagem de sistemas.

Um dos grandes conceitos na linguagem é a herança, ela permite que uma classe, subclasse ou classe derivada herde atributos e métodos de outra classe (ou seja, superclasse ou classe base). Isso promove a reutilização de código e a criação de hierarquias de classes. A palavra-chave `super` é usada para acessar membros da superclasse. Outra grande invenção no Java é o polimorfismo, que, como você já sabe, significa “muitas formas”. Ele permite que objetos de diferentes classes sejam tratados por meio de uma interface comum, superclasse ou interface. Em Java, manifesta-se principalmente através da sobrescrita de métodos, ou como é comumente conhecida, `overriding`, na qual uma subclasse fornece uma implementação específica para um método herdado. Há também o conceito de sobrecarga de métodos, chamado de `overloading`, em que múltiplos métodos com o mesmo nome, mas com assinaturas diferentes, coexistem na mesma classe.

Temos ainda as classes abstratas, que não podem ser instanciadas diretamente e podem conter métodos abstratos, ou seja, sem implementação. Servem como modelos para subclasses, que devem implementar os métodos abstratos.

Por outro lado, as interfaces definem um contrato de métodos que uma classe pode implementar. Uma classe pode implementar múltiplas interfaces, permitindo uma forma de herança múltipla de tipos, mas não de implementação. As interfaces contêm apenas assinaturas de métodos e constantes.

Leitura

Momento de Reciclagem de Conhecimento

Inheritance (Oracle Docs)

Clique no botão para conferir o conteúdo.

ACESSE

Polymorphism (Oracle Docs)

Clique no botão para conferir o conteúdo.

ACESSE

Abstract Methods and Classes (Oracle Docs)

Clique no botão para conferir o conteúdo.

ACESSE

Interfaces (Oracle Docs)

Clique no botão para conferir o conteúdo.

ACESSE

Para armazenar e manipular grupos de objetos ou dados primitivos, o Java oferece estruturas de dados como arrays e o Java Collections Framework.

Arrays são estruturas de tamanho fixo que armazenam elementos do mesmo tipo. Podem ser unidimensionais ou multidimensionais. O acesso aos elementos é feito por um índice numérico. Por outro lado, Java Collections Framework é um conjunto robusto de classes e interfaces, como *List*, *Set* e *Map*, utilizados para representar e manipular coleções de objetos de forma mais flexível e eficiente que as arrays ou matrizes.

Leitura

Arrays (Oracle Docs)

Clique no botão para conferir o conteúdo.

ACESSE

Java Collections Framework Overview (Oracle Docs)

Clique no botão para conferir o conteúdo.

ACESSE

Vamos lembrar o que são as exceções: elas se referem a eventos anormais que ocorrem durante a execução de um programa e interrompem o fluxo normal de instruções. O tratamento adequado de exceções é crucial para a construção de um software resiliente, ou seja, quando um erro ocorre — por exemplo, uma divisão por zero ou acesso a um arquivo inexistente —, uma exceção é lançada. Chamamos esse evento de *throw*.

Para isso lidar com isso, utilizamos os blocos try-catch-finally da seguinte forma:

- **Try:** envolve o bloco de código onde uma exceção pode ocorrer;
- **Catch:** captura e trata uma exceção específica que foi lançada no bloco try. Pode haver múltiplos blocos catch para diferentes tipos de exceção;
- **Finally:** bloco de código que é sempre executado, independentemente de uma exceção ter sido lançada ou capturada. Utilizado para liberar recursos (ex: fechar arquivos ou conexões de banco de dados).

O tratamento de exceções permite que o programa lide com erros de forma elegante, evitando que ele termine abruptamente e fornecendo feedback útil ao usuário ou registrando o erro para análise posterior.

Leitura

Momento de Aprendizagem

Lesson: Exceptions (Oracle Docs)

Clique no botão para conferir o conteúdo.

ACESSE

Site

Java Try Catch (W3Schools)

Clique no botão para conferir o conteúdo.

ACESSE

Projeto do Sistema e Segurança da Informação

A ideia por trás desta seção é mostrar as fases mais importantes do ciclo de vida de um projeto de software, desde a concepção e o planejamento detalhado, passando pelo desenvolvimento iterativo com atenção aos aspectos de segurança, até a documentação técnica e a preparação para a apresentação final da solução computacional desenvolvida.

Saiba que, antes de qualquer linha de código ser escrita, um planejamento cuidadoso e uma modelagem precisa do sistema são etapas imperativas. Segundo Pressman e Maxim (2016, p. 78), “o planejamento de projeto de software envolve a estimativa de esforço, custo e cronograma, bem como o gerenciamento de riscos”. Embora nosso escopo seja reduzido, a aplicação desses princípios é fundamental.

O ponto de partida é a clara compreensão do problema a ser resolvido e dos objetivos do projeto. Para o nosso Catálogo Simples de Livros, Séries ou Filmes, os requisitos funcionais mandatórios já foram estabelecidos.

- Interface web para navegação e gerenciamento;
- Cadastro de novos itens (livros, séries, filmes);
- Listagem de todos os itens catalogados;
- Visualização dos detalhes de um item específico;
- Edição das informações de um item;
- Exclusão de um item do catálogo;
- Busca simples por título ou autor/diretor;
- Persistência dos dados em banco de dados.

Nesta fase, você deve refinar o escopo, o que inclui confirmar o entendimento de cada funcionalidade. Deve decidir se o catálogo será focado em apenas um tipo de mídia livros ou se permitirá múltiplos livros e filmes, por exemplo. Para simplificação inicial, focar em uma única entidade, como “ItemDeMidia” com um campo “tipo” (livro, filme, série), pode ser uma abordagem útil.

É necessário levantar requisitos não funcionais básicos. Para tanto, considere aspectos como usabilidade, por exemplo, uma interface simples e intuitiva, e desempenho, que pode ser relacionado a respostas rápidas para cadastro e busca, dentro do razoável para um projeto acadêmico.

É importante, para seu controle, criar um cronograma de entregas parciais. Como foi sugerido o método ágil, acredito que você deva organizar as *sprints* ou iterações e as entregas esperadas. Portanto, divida o projeto em marcos menores, como: módulo de cadastro, módulo de listagem/busca, módulo de edição/exclusão. Isso facilita o acompanhamento e permite feedbacks iterativos, ajudando você mesmo a perceber a evolução do que está desenvolvendo.

A modelagem visualiza a estrutura e o comportamento do sistema antes de sua implementação. Fowler (2004, p. 15) argumenta que a modelagem é crucial para “compreender o sistema a ser desenvolvido e comunicar essa compreensão entre os membros da equipe”. Você utilizará, para tanto:

- **Diagrama de Casos de Uso da UML:** para identificar os atores e as principais interações que eles terão com o sistema;
- **Diagrama de Classes da UML:** para representar a estrutura estática do sistema. A classe principal será, por exemplo, ItemMidia, com atributos como id (inteiro, chave primária), titulo (*String*), autorDiretor (*String*), anoLancamento (inteiro), genero (*String*), sinopse (*String*), tipoMidia (*String*). Métodos associados a essa classe podem incluir getters e setters para seus atributos. Poderá haver também classes de serviço e classes DAO;

Glossário

UML: sigla para Unified Modeling Language (Linguagem de Modelagem Unificada). Linguagem gráfica para especificar, visualizar, construir e documentar artefatos de sistemas de software.

- **Modelagem do Banco de Dados:** pode, para isso, utilizar o Diagrama Entidade-Relacionamento (DER), que definirá a estrutura das tabelas no banco de dados. Para nosso projeto, por exemplo, uma tabela `item_midia`, com colunas correspondentes aos atributos da classe `ItemMidia`, será suficiente.

A seguir, disponibilizamos um exemplo dessa modelagem:

```
Tabela: item_midia
  id INT PRIMARY KEY AUTO_INCREMENT (ou SERIAL)
  titulo VARCHAR(255) NOT NULL
  autor_diretor VARCHAR(255)
  ano_lancamento INT
  genero VARCHAR(100)
  sinopse TEXT
  tipo_midia VARCHAR(50) NOT NULL (ex: 'Livro', 'Filme', 'Série')
```

Introdução aos Fundamentos de Segurança da Informação

Mesmo em projetos simples, a segurança da informação não pode ser negligenciada. A ISO/IEC 27001:2013 define que a segurança da informação visa proteger os ativos de informação contra uma ampla gama de ameaças. Para nosso projeto do catálogo, os principais ativos são os dados dos itens catalogados. Os pilares da segurança — confidencialidade, integridade e disponibilidade — devem ser considerados

Por exemplo, a confidencialidade, em nosso projeto, não há dados sensíveis de usuários. Porém, se houvesse um sistema de login, a proteção das credenciais seria crucial. A integridade, por sua vez, serve para garantir que os dados do catálogo (título, autor etc.) sejam precisos e não sejam modificados indevidamente. Isso se relaciona com a validação de

entradas. Por fim, o sistema deve estar acessível para consulta e gerenciamento quando necessário.

Você Sabia?

No contexto do desenvolvimento, McGraw (2006, p. 18) advoga pelo “*Secure by Design*”, ou seja, pensar em segurança desde o início. As ameaças mais comuns em aplicações web simples que devemos ter em mente para o catálogo incluem:

- **SQL Injection:** manipulação de entradas para executar comandos SQL maliciosos no banco de dados;
- **Cross-Site Scripting (XSS):** injeção de scripts maliciosos em páginas web visualizadas por outros usuários (menos crítico em nosso projeto se não houver exibição de dados inseridos por um usuário para outro sem tratamento, mas importante para a sua conscientização).

Você deverá ter em mente a mitigação dessas ameaças durante o desenvolvimento.

Com o planejamento e a modelagem concluídos, inicia-se o ciclo de desenvolvimento. Adotaremos uma abordagem iterativa, construindo e testando o sistema em partes, de tal forma que cada funcionalidade, o CRUD e a busca serão implementados com a utilização de:

- **Java:** será utilizado para a lógica de negócios, aplicando classes de modelo como ItemMidia, classes de serviço (se necessário) e classes DAO (sigla para Data Access Object), que encapsularão o acesso ao banco de dados;
- **Servlets:** atuarão como controladores, recebendo requisições **HTTP** das páginas JSP, processando-as, invocando a lógica de negócios e os DAOs, e encaminhando a resposta de volta para uma JSP;
- **JSP (sigla para JavaServer Pages):** serão responsáveis pela camada de apresentação (View), exibindo os formulários de cadastro/edição e as listagens de itens. Utilizarão Expression Language (EL) e a JavaServer Pages Standard Tag Library (JSTL) para simplificar a exibição de dados e a lógica na view;
- **Java Database Connectivity (JDBC):** será usado para conectar a aplicação Java ao banco de dados relacional, como MySQL, PostgreSQL etc., e executar as instruções SQL de inserção, seleção, atualização e exclusão dos itens do catálogo.

Glossário

HTTP: sigla para Hypertext Transfer Protocol. Protocolo de comunicação usado para a transferência de informações na World Wide Web.

Você Sabia?

Para facilitar seu entendimento construímos um exemplo de fluxo para Cadastrar Item.

- **JSP (cadastroItem.jsp):** exibe um formulário HTML;
- Usuário preenche e submete o formulário;
- **Servlet (CadastrarItemServlet):** recebe os dados do formulário via `request.getParameter()`;
- **Servlet:** valida os dados recebidos;
- **Servlet:** cria um objeto `ItemMidia` com os dados;
- **Servlet:** invoca um método em `ItemMidiaDAO`, por exemplo, `inserir(ItemMidia item)` para persistir o objeto no banco;
- **ItemMidiaDAO:** utiliza JDBC – `PreparedStatement` para executar o `INSERT SQL`;
- **Servlet:** redireciona para uma página de sucesso ou para a listagem de itens.

Durante a implementação, aplicaremos práticas básicas de segurança, como:

- **Prevenção de SQL Injection:** utilizar `PreparedStatement`. Essa é a principal defesa contra ataques de SQL Injection em aplicações Java que usam JDBC. As `PreparedStatement`s pré-compilam as instruções SQL e tratam os parâmetros de entrada como dados, e não como parte executável do comando SQL.

Vejamos um exemplo de código em `ItemMidiaDAO`:

```

String sql = "INSERT INTO item_midia (titulo, autor_diretor,
ano_lancamento, genero, sinopse, tipo_midia) VALUES (?, ?, ?, ?, ?, ?)";
try (Connection conn = FabricaDeConexoes.getConexao()); // Método para
obter conexão
    PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setString(1, item.getTitulo());
    stmt.setString(2, item.getAutorDiretor());
    // ... setar outros parâmetros
    stmt.executeUpdate();
} catch (SQLException e) {
    // Tratar exceção
}

```

Lembrando que se trata de apenas um exemplo; ele não rodará se você copiar e colar, por isso é importante entender a lógica do que estamos escrevendo por aqui.

Antes de processar qualquer dado vindo do cliente, é preciso fazer a seguinte validação:

- Verificar se campos obrigatórios não estão vazios;
- Verificar tipos de dados, por exemplo, se anoLancamento deve ser um número;
- Verificar comprimentos máximos para evitar estouro de buffer no banco, embora você possa codar isso no **SGBD** como restrição.

Outra dica importante: ao exibir dados no JSP que foram inseridos pelo usuário, como título e sinopse, você deverá garantir que sejam devidamente "escapados" para que qualquer HTML/JavaScript inserido seja tratado como texto literal e não executado. A JSTL com a tag `<c:out value="${item.titulo}" />` faz isso por padrão. Se não usar JSTL para saída, é preciso ter atenção.

Temos que testar sempre! Myers, Sandler e Badgett (2012, p. 4) enfatizam que "o teste de software é um processo de execução de um programa com o intuito de encontrar erros". À

medida que cada módulo ou funcionalidade é desenvolvida, testes devem ser realizados. Você poderá utilizar o JUNIT para isso:

- **Testes Unitários ou Conceituais:** idealmente, cada método crítico, especialmente nos DAOs e classes de serviço, deveria ser testado isoladamente. Para um projeto inicial, pode-se focar em testes manuais mais rigorosos de cada funcionalidade;
- **Testes de Integração ou Funcional:** você deverá testar o fluxo completo, desde a submissão de um formulário no JSP até a verificação dos dados no banco e sua correta exibição. Por exemplo, cadastrar um livro e verificar se ele aparece corretamente na listagem e se a busca por seu título o encontra.

Não esqueçamos a documentação, porque ela não é uma reflexão tardia, mas, sim, um componente integral do desenvolvimento. Ela facilita a compreensão, manutenção e evolução do software. A apresentação final é a oportunidade de demonstrar o trabalho realizado por você e as competências adquiridas.

Portanto, aproveitamos para te informar que a documentação do nosso projeto para o Catálogo Simples de Livros, Séries ou Filmes deverá incluir um Relatório Técnico do Projeto, com os seguintes conteúdos:

- **Introdução:** apresentação do projeto, objetivos e justificativa (mesmo que seja acadêmica);
- **Metodologia:** breve descrição da abordagem de desenvolvimento — que, no nosso caso, será a iterativa — e das tecnologias utilizadas, lembrando que, em nosso projeto, teremos Java, Servlets, JSP, JDBC e o SGBD escolhido por você;
- **Modelagem do sistema:** incluir os diagramas de casos de uso, diagrama de classes principal e o esquema do banco de dados — nesse caso, os DER e o script SQL de criação da tabela ou das tabelas;
- **Desenvolvimento e implementação:** descrever a arquitetura geral da aplicação que você criará, por exemplo, Servlets, JSPs e DAOs, e detalhar a implementação de uma ou duas funcionalidades-chave, por exemplo, Cadastro e Busca, mostrando trechos de código relevantes, como os Servlets, JSPs e DAOs, e a explicando da lógica;

- **Resultados:** apresentar as telas, ou seja, produzir alguns screenshots da aplicação funcional, demonstrando as funcionalidades implementadas;
- **Considerações sobre segurança:** descrever as medidas de segurança que foram implementadas;
- **Conclusões:** dificuldades encontradas, aprendizados e possíveis trabalhos futuros.
- **Referências:** listar todas as fontes consultadas para o trabalho — livros, artigos, documentações on-line e cursos ou vídeos;
- **Documentação do código-fonte:**
 - Comentários claros e concisos no código Java, explicando a lógica de blocos complexos, o propósito de classes e métodos. O uso de **Javadoc** para os métodos públicos das classes DAO e Servlets é uma boa prática;
- **Manual do usuário simplificado:**
 - Um pequeno guia contendo uma ou duas páginas explicando como instalar se houver passos específicos além de *deploy* no servidor e como utilizar as funcionalidades básicas do catálogo.

Ao final, você deverá entregar tudo isso nas seguintes modalidades:

- Documentação em arquivo de programa da linguagem-fonte;
- Documentação e relatório técnico em arquivo no formato PDF, submetido ao(à) tutor(a) para avaliação.

Glossário

- **SGBD**: sigla para Sistema Gerenciador de Banco de Dados. Software que permite criar, gerenciar e manipular bancos de dados (ex: MySQL, PostgreSQL);
- **Javadoc**: ferramenta para gerar documentação HTML a partir de comentários especiais no código Java.

Em Síntese

A disciplina de Aprendizagem Baseada em Projetos (ABP) é uma estratégia pedagógica que coloca o(a) aluno(a) no centro do processo, incentivando a aquisição de conhecimento e habilidades por meio da investigação ativa e resolução de problemas do mundo real. O(A) professor(a) atua como facilitador(a), enquanto a abordagem enfatiza a autonomia, a colaboração e a reflexão.

No contexto da Ciência da Computação, a ABP é particularmente vantajosa, promovendo o desenvolvimento de habilidades técnicas aplicadas, competências socioemocionais (*soft skills*), aumento da motivação e engajamento, além de uma aprendizagem profunda e significativa que prepara o(a) estudante para os desafios do mercado de trabalho. Nesse modelo, o(a) aluno(a) assume um papel ativo, investigativo, colaborativo, autônomo e reflexivo.

A disciplina de PIT (Projeto Integrador Transdisciplinar) em Ciência da Computação baseia-se integralmente na ABP, tendo como objetivo a integração de competências e o desenvolvimento de soluções computacionais aplicáveis a problemas reais. O projeto principal é a criação de um Catálogo Simples de Livros ou Filmes, utilizando Java e Programação Orientada a Objetos

(POO). As funcionalidades incluem CRUD (Cadastro, Listagem, Visualização, Edição, Exclusão), além de uma busca simples, com persistência de dados em banco.

O desenvolvimento do projeto segue as fases: concepção e planejamento, desenvolvimento iterativo, testes e refinamento, e documentação e preparação para apresentação. A segurança da informação, como a prevenção de SQL Injection, é abordada desde o início do projeto. A disciplina também reforça os fundamentos de Java (sintaxe, POO, coleções e tratamento de exceções) e a importância das habilidades do século XXI (pensamento crítico, criatividade, colaboração e comunicação).

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta
Unidade:

Sites

SPRING.IO. Spring Boot Documentation.

A documentação oficial do Spring Boot é a fonte mais completa e atualizada para aprender a desenvolver aplicações web com este popular framework Java. Oferece guias, tutoriais passo a passo (Getting Started Guides) e referências detalhadas. Embora em inglês, é um recurso indispensável. Muitos tutoriais cobrem desde a configuração inicial até a criação de APIs RESTful.

Clique no botão para conferir o conteúdo.

ACESSE

Baeldung

É uma excelente fonte de tutoriais e exemplos práticos sobre Spring Boot, frequentemente citado e recomendado pela comunidade de desenvolvedores Java.

Clique no botão para conferir o conteúdo.

ACESSE

DevSuperior

Canal no YouTube que possui tutoriais completos e práticos sobre Git e GitHub para iniciantes além de JAVA.

Clique no botão para conferir o conteúdo.

ACESSE

Vídeo

GitHub Como Usar – Tutorial para Iniciantes – Guia Completo e Atualizado

Nesse vídeo você vai aprender do absoluto zero a trabalhar com o Git e GitHub.

Clique no botão para conferir o vídeo indicado.

ASSISTA

Situação-Problema 1

Caro(a), estudante.

Agora, vamos compreender o cenário que será abordado na primeira situação-problema de caso da disciplina.

Atente-se à situação profissional que você precisará entender para poder realizar a atividade.

Modelagem e Segurança

Um aluno está desenvolvendo o Catálogo Simples de Livros/Filmes e decide criar uma única tabela no banco de dados chamada `itens` com colunas como `id`, `titulo`, `autor`, `ano`, `genero` e `detalhes`. Para permitir buscas mais flexíveis, o aluno concatena os campos `titulo` e `autor` em uma única string para a busca. Ao implementar a funcionalidade de busca, ele percebe que está vulnerável a SQL Injection.

- Quais são os riscos de segurança ao concatenar campos diretamente em consultas SQL?
- Como o uso de `PreparedStatement` pode mitigar essa vulnerabilidade? Demonstre com um exemplo de código.

- Como a modelagem do banco de dados pode influenciar na segurança e eficiência da aplicação?

CONTINUE

- Concatenar campos diretamente em consultas SQL permite que um atacante insira código SQL malicioso através das entradas do usuário, comprometendo a integridade e confidencialidade dos dados;
- PreparedStatement impede a interpretação das entradas do usuário como código SQL, tratando-as como dados. Exemplo:

```
String sql = "SELECT * FROM itens WHERE titulo LIKE ? OR autor LIKE ?";
PreparedStatement stmt = conn.prepareStatement(sql);
stmt.setString(1, "%" + termoDeBusca + "%");
stmt.setString(2, "%" + termoDeBusca + "%");
ResultSet rs = stmt.executeQuery();
```

- Uma modelagem inadequada pode levar a vulnerabilidades e ineficiências. A normalização do banco de dados e a escolha correta dos tipos de dados podem melhorar a segurança e o desempenho.

Situação-Problema 2

Caro(a), estudante.

Agora, vamos compreender o cenário que será abordado na segunda situação-problema de caso da disciplina.

Atente-se à situação profissional que você precisará entender para poder realizar a atividade.

Tratamento de Exceções e Validação de Dados

Durante o desenvolvimento da funcionalidade de cadastro de livros, o aluno não implementa tratamento de exceções ao tentar inserir um novo livro no banco de dados. Além disso, não há validação dos dados inseridos pelo usuário (e.g., o campo ano pode receber *strings*). Quando um usuário tenta cadastrar um livro com um ano inválido, a aplicação quebra, exibindo uma mensagem de erro genérica.

- Por que é importante tratar exceções em aplicações Java?

- Como os blocos try-catch-finally podem ser utilizados para tratar a exceção de inserção de dados inválidos?
- Quais tipos de validações de dados devem ser implementadas no formulário de cadastro e como elas podem prevenir erros?

CONTINUE

- O tratamento de exceções garante que a aplicação não termine abruptamente em caso de erros, proporcionando uma melhor experiência ao usuário e permitindo o registro de informações de depuração.
- O bloco try envolve o código que pode lançar uma exceção (e.g., a inserção no banco de dados). O bloco catch captura a exceção e permite o tratamento adequado (e.g., exibir uma mensagem de erro amigável). O bloco finally garante que recursos sejam liberados (e.g., fechar conexões com o banco de dados), independentemente de ocorrer ou não uma exceção.
- Validações como verificar se campos obrigatórios estão preenchidos, validar o tipo de dados (e.g., ano deve ser um número), e verificar comprimentos máximos previnem erros e vulnerabilidades.

Situação-Problema 3

Caro(a), estudante.

Agora, vamos compreender o cenário que será abordado na terceira situação-problema da disciplina.

Atente-se à situação profissional que você precisará entender para poder realizar a atividade.

Refatoração e Orientação a Objetos

O aluno implementa todas as funcionalidades do Catálogo Simples (CRUD e busca) diretamente na Servlet, resultando em um código extenso e difícil de manter. Há duplicação de código em várias partes da aplicação, especialmente no acesso ao banco de dados.

- Como a aplicação dos Princípios de Orientação a Objetos (POO) pode melhorar a estrutura e a manutenibilidade do código?
- Qual o papel das classes *DAO* (*Data Access Object*) na separação de responsabilidades e como elas podem reduzir a duplicação de código?
- Como a refatoração do código para utilizar classes de serviço pode melhorar a organização e testabilidade da aplicação?

CONTINUE

- POO promove a modularidade, reutilização e manutenibilidade do código. Encapsulamento, herança e polimorfismo permitem criar um código mais organizado e fácil de entender;
- DAOs encapsulam a lógica de acesso a dados, isolando-a do restante da aplicação. Isso facilita a manutenção e permite que a camada de persistência seja alterada sem afetar outras partes do sistema;
- Classes de serviço implementam a lógica de negócios da aplicação, separando-a da Servlet e das DAOs. Isso torna o código mais organizado, testável e fácil de manter.

Atividade de Entrega

Muito bem, estudante!

Agora que você já leu todas as situações-problema, você pode acessar o Ambiente Virtual de Aprendizagem para realizar a atividade de entrega.

Referências

BENDER, W. N. **Aprendizagem baseada em projetos**: educação diferenciada para o século XXI. 1. ed. Porto Alegre: Penso, 2014.

BLUMENFELD, P. C. *et al.* *Motivating project-based learning: Sustaining the doing, supporting the learning*. **Educational Psychologist**, [s.l.], v. 26, n. 3-4, p. 369-398, 1991. Disponível em: <<https://doi.org/10.1080/00461520.1991.9653139>>. Acesso em: 05/04/2025.

BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. **UML**: guia do usuário. 2. ed. Rio de Janeiro: Elsevier, 2007.

DEITEL, P.; DEITEL, H. **Java®**: como programar. 10. ed. São Paulo: Pearson Universities, 2017.

FOWLER, M. **UML distilled**: a brief guide to the standard object modeling language. 3rd ed. Boston: Addison-Wesley Professional, 2004.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 27001:2013**: *Information technology – Security techniques – Information security management systems – Requirements*. 1st ed. Geneva: ISO, 2013.

KNUTH, D. E. *Computer programming as an art*. **Communications of the ACM**, v. 17, n. 12, p. 667-673, Dec. 1974. Disponível em: <<https://doi.org/10.1145/361604.36161>>. Acesso em: 05/04/2025.

LARMER, J.; MERGENDOLLER, J.; BOSS, S. ***Setting the standard for project-based learning: a proven approach to rigorous classroom instruction***. 1st ed. Alexandria: ASCD, 2015.

MCGRRAW, G. ***Software security: building security in***. 1st ed. Upper Saddle River: Addison-Wesley, 2006.

MYERS, G. J.; SANDLER, C.; BADGETT, T. ***The art of software testing***. 3rd ed. Hoboken: John Wiley & Sons, 2012.

OLIVEIRA, K.; Aprendizagem Baseada em Projetos como estratégia para o desenvolvimento de atividades não presenciais no ensino médio integrado em informática no IFB Campus Brasília. **Revista Brasileira da Educação Profissional e Tecnológica**, v. 2, n. 22, e11815, 2022 (CC BY 4.0) | ISSN 2447-1801 | DOI: <<https://doi.org/10.15628/rbept.2022.11815>>. Acesso em: 05/04/2025.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.

THOMAS, J. W. ***A review of research on project-based learning***. 1st ed. San Rafael: Autodesk Foundation, 2000. Disponível em: <http://www.bobpearlman.org/BestPractices/PBL_Research.pdf>. Acesso em: 25/05/2025.

TRILLING, B.; FADEL, C. ***21st century skills: learning for life in our times***. 1st ed. San Francisco: Jossey-Bass, 2009.



Muito bem, estudante! Você concluiu o material de estudos! Agora,

volte ao Ambiente Virtual de Aprendizagem para realizar a Atividade.